neo4j

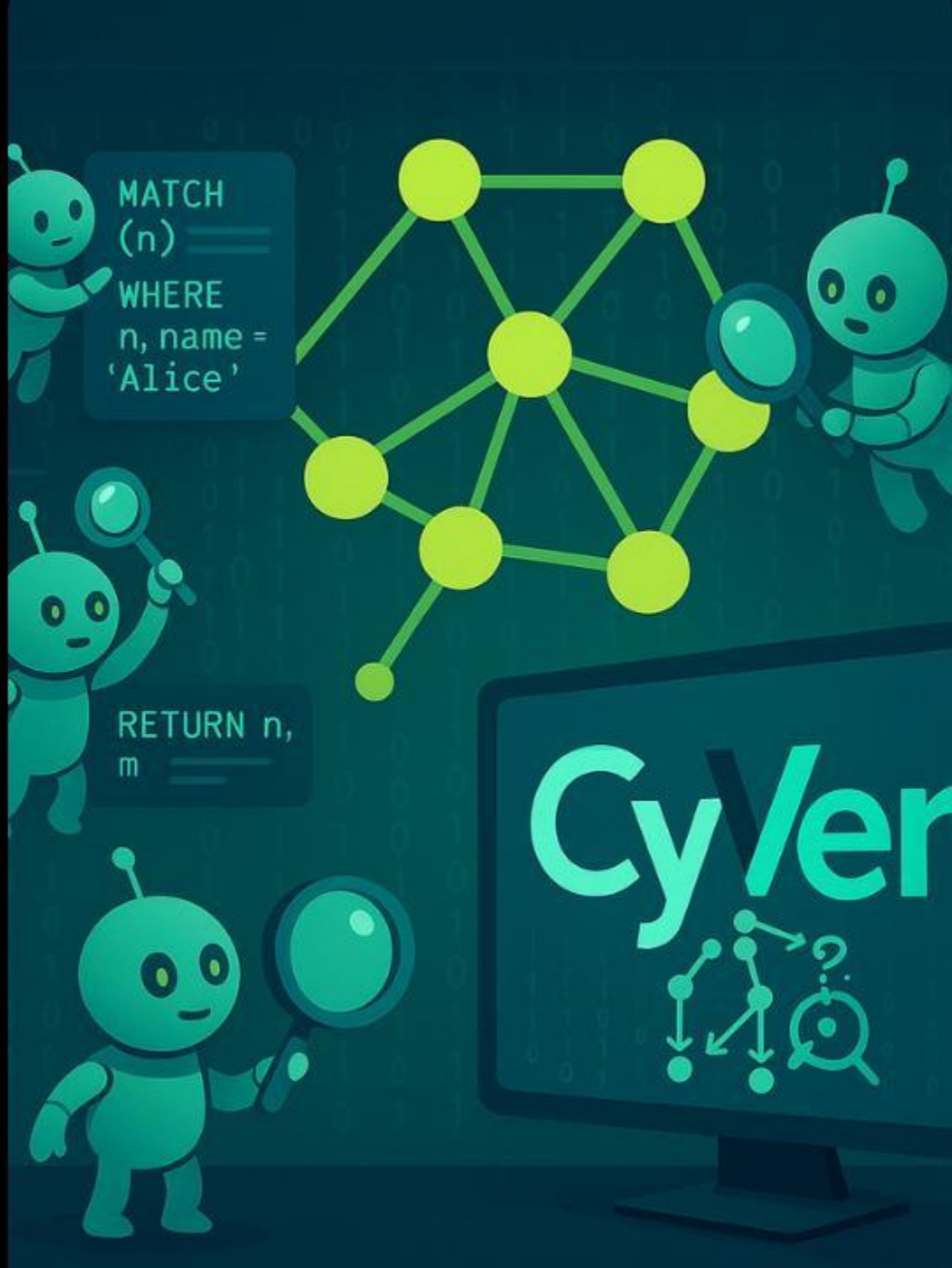
CyVer - Verifying Cypher Queries





Christina-Maria Androna
androna xm@netmode.ntua.gr

Ioanna Mandilara
ioannamand@netmode.ntua.gr





Institute of Communication and Computer Systems



National Technical University of Athens



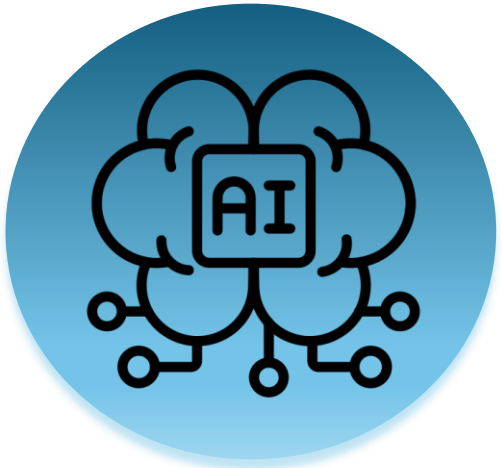
https://www.netmode.ntua.gr/



https://arsinoe-project.eu/

European Union’s Horizon H2020 innovation action programme





neo4j

The problem: Text to Cypher Task



LLMs are generating queries. Can we trust them?

Are LLMs reliable enough to generate correct Cypher queries without human feedback?



What happens when they hallucinate?
How often do LLMs make up non-existent properties, labels, or relationships in Cypher queries?



Do they understand the schema, properties and syntax of the Cypher queries?

Or are they just predicting plausible patterns without true comprehension?

Motivation

Enable non-technical users to interact with a graph database

Increase accessibility and usability of graph dbs.

Evaluation of **open-source** models to the Text-To-Cypher task

Open-source models are transparent, customizable & cost-effective.

Existing metrics primarily assess string similarity or Cypher output similarity.

Effective evaluation should prioritize functional validity.

BLEU score
Rouge-L,
Jaccard/ Jaro similarity
Pass@1,
Pass@k(Exact Match)



Guidance of LLMs to improve their answers

Should evaluations consider error types such as syntax correctness, schema conformity, and property validity?



Open-source python library designed to validate Cypher queries against a given Knowledge Graph schema in Neo4j.

It ensures correctness in terms of **syntax**, **schema** validity, and **property** access.

Use regex patterns to detect node, relationships, paths and accessed properties of cypher queries.

Use **path-pattern inference** to detect node labels or relationship types .

GitLab

python Package Index

What CyVer offers?

Main Classes:

1. **SyntaxValidator**
   - *Validate* : Returns a Boolean variable indicating if the query is syntactically correct
2. **SchemaValidator**
   - *Extract* : Returns a list of the extracted nodes, relationship and path patterns
   - *Validate*: Returns a score in [0,1]
3. **PropertiesValidator**
   - *Extract*: Returns two dictionaries of the mapping of the variables with the accessed properties and the mapping of the labels with the accessed properties respectively.
   - *Validate*: Returns a score in [0,1]

Syntax Validator: Validate Method

Validate method: Validates Cypher queries for correct syntax and detects conflicting labels in nodes and relationships.



CORRECT

```
MATCH (g:Goal)
RETURN COUNT(g) AS output

MATCH (g:Goal)-[:HAS_TARGET]->(t:Target)
RETURN COUNT(g) AS output
```



CONFLICTING LABELS

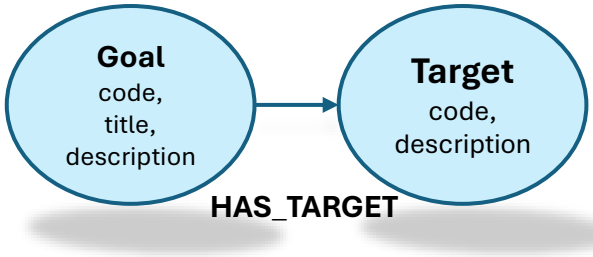
```
MATCH (g:Goal)
MATCH (g:Target)
RETURN COUNT(g) AS output
```

UNDEFINED VARIABLE

```
MATCH (g:Goal)-[:HAS_TARGET]->(t:Target{code:$w})
RETURN COUNT(g) AS output
```



Schema Validator: Extract Method



Returns:

- **Node patterns**

[(:nodeLabel)]

- **Relationship patterns :**

[()-[:relType]->()]

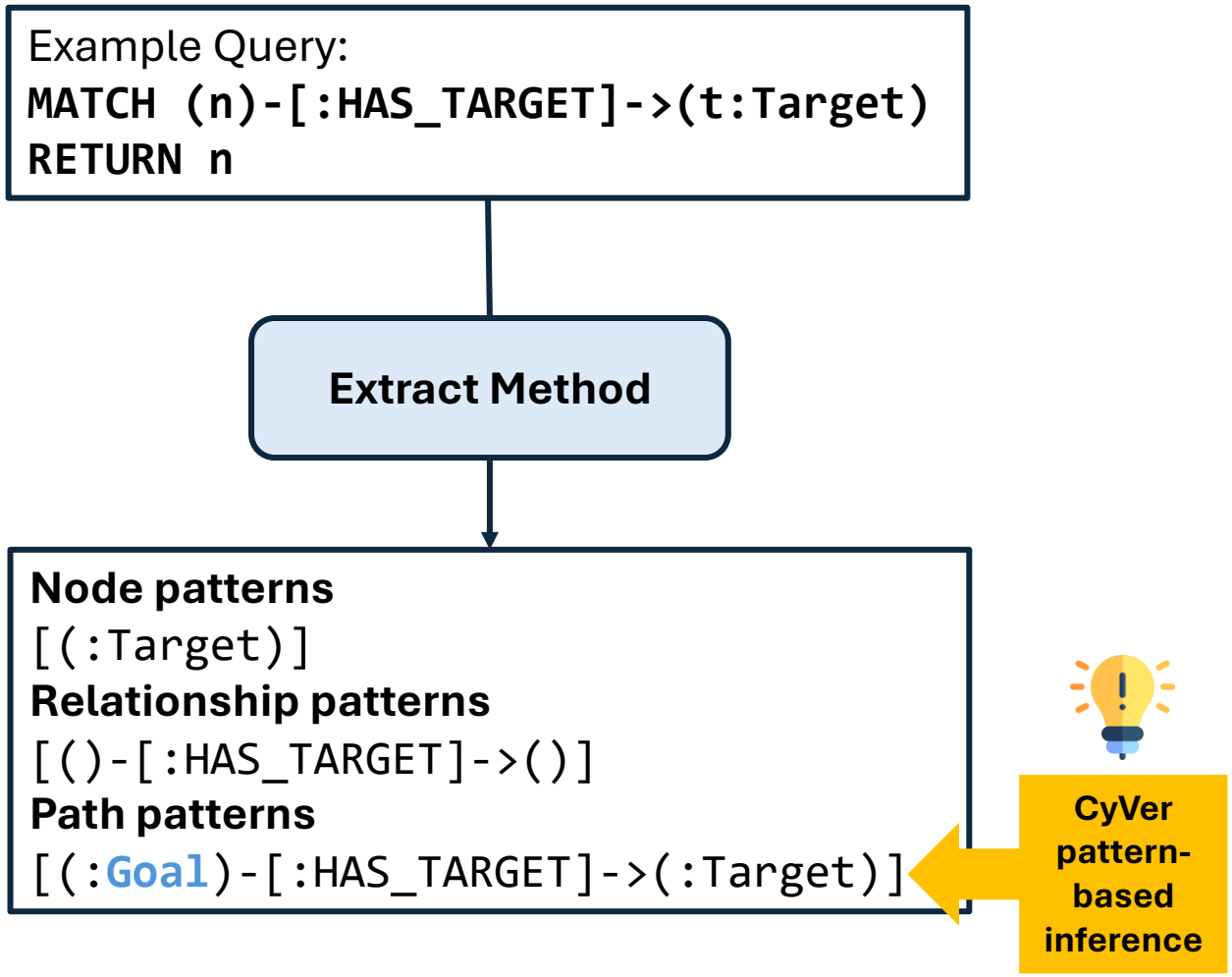
- **Path patterns :**

[(:nodeLabel1)-[:relType]->(:nodeLabel2)]

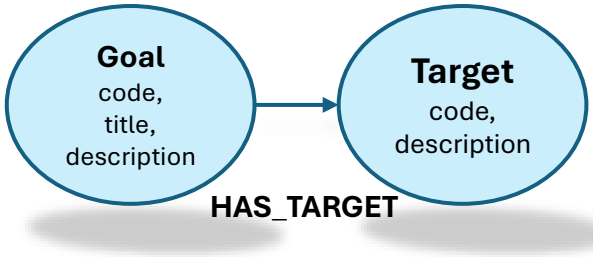
Pattern-based inference

When node labels or relationship types are missing in a Cypher query...

➡️ CyVer uses **pattern-based inference** to guess missing node labels or relationship types using the schema in the extracted path patterns.



Schema Validator: Validate Method



Returns: A score in [0,1] reflecting the average of the non-empty valid extracted node, relationship and path patterns

$$Score = w_1 \times \left(\frac{Valid\ extracted\ node\ patterns}{Total\ extracted\ node\ patterns}\right) + w_2 \times \left(\frac{Valid\ extracted\ relationship\ patterns}{Total\ extracted\ relationship\ patterns}\right) + w_3 \times \left(\frac{Valid\ extracted\ path\ patterns}{Total\ extracted\ path\ patterns}\right)$$

$where\ w_1, w_2, w_3$: weighting factors that depend on the extracted components

Example Query 1 :

```
MATCH (n)-[:HAS_TARGET1]->(t:Target)
RETURN n
```

Extract Method

Node patterns
`[(:Target)]`
Relationship patterns
`[()-[:HAS_TARGET1]->()]`
Path patterns
`[()-[:HAS_TARGET1]->(:Target)]`

Validate Method

1/3 (1/1) +
1/3(0/1) +
1/3 (0/1) =**0.33**

Example Query 2 :

```
MATCH (:Goal)-[]->(t:Target)
RETURN t
```

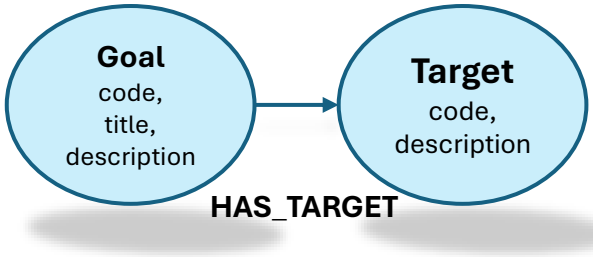
Extract Method

Node patterns
`[(:Goal), (:Target)]`
Relationship patterns
`[]`
Path patterns
`[(:Goal)-[:HAS_TARGET]->(:Target)]`

Validate Method

1/2 (2/2) +
0+
1/2 (1/1) =**1**

Properties Validator: Extract Method



Returns:

- **Variables-Properties Mapping**

Var_x → [property1, property2]

- **Labels-Properties Mapping:**

nodeLabel/ relType → [property1, property2]

Pattern-based inference

When variables access properties in a Cypher query…

➡️ CyVer uses **pattern-based inference** to guess their node labels or relationship types considering also the accessed properties.

Example Query:

```
MATCH (n)-[:HAS_TARGET]->(t:Target{code:'1.1'})
WHERE n.title ='End poverty in all its forms everywhere'
RETURN n
```

Extract Method

Variables-Properties Mapping

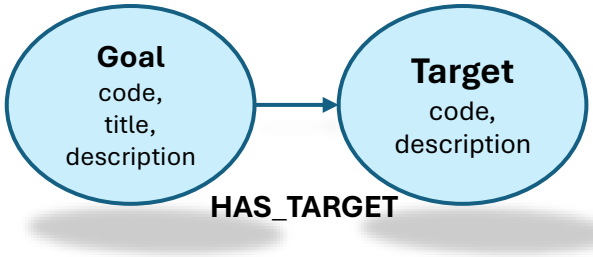
```
{
n: [title]
}
```

Labels-Properties Mapping

```
{
Target: [code],
Goal:  [title]
}
```

CyVer pattern-based inference

Properties Validator: Validate Method



Returns: A score in [0,1] indicating the percentage of correctly accessed properties.

$$Score = \begin{cases} \dfrac{correctly\ accesed\ properties}{total\ accesed\ properties}, & if\ total\ accesed\ properties > 0 \\ None, & if\ total\ accesed\ properties = 0 \end{cases}$$

Example Query 1 :

```
MATCH (n)-[:HAS_TARGET]->(t:Target)
WHERE n.title1 ="1" AND t.code='1.1'
RETURN n
```

Extract Method

Variables-Properties Mapping
{n: [title1]}
Labels-Properties Mapping
{Target: code,
 Goal: title1}

Validate Method

1/2=**0.5**

Example Query 2 :

```
MATCH (:Goal)-[]->(t:Target)
RETURN t
```

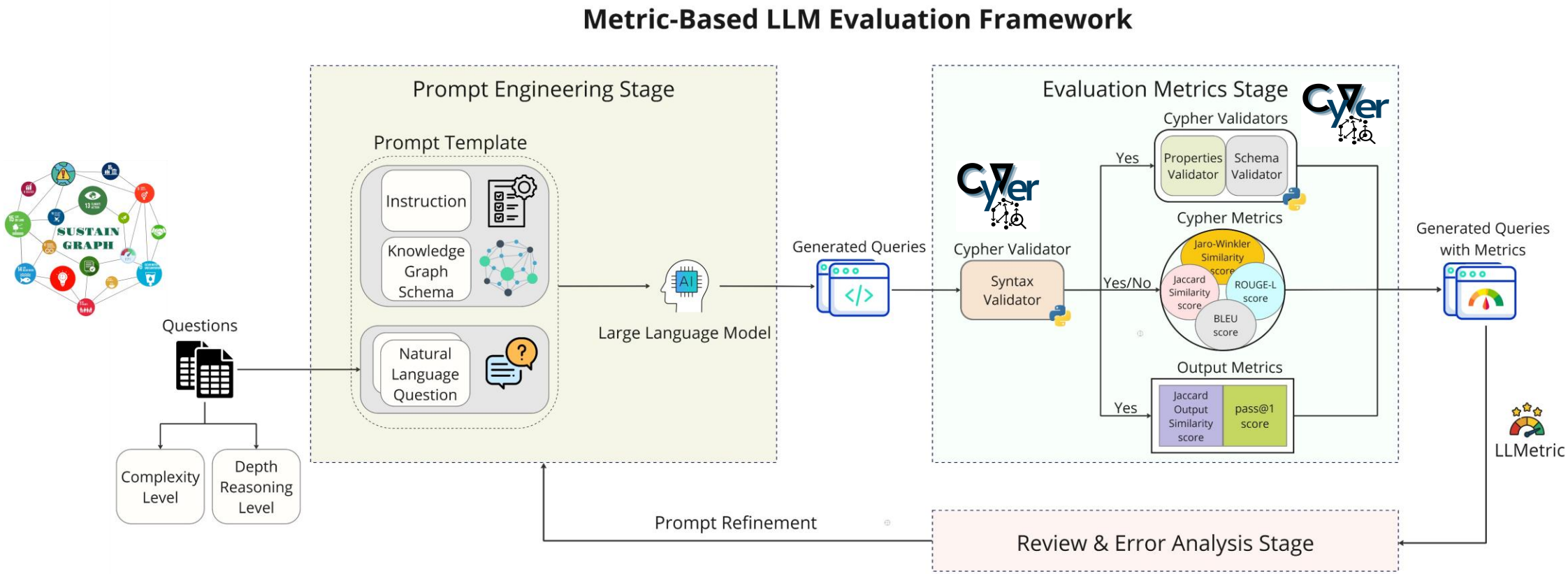
Extract Method

Variables-Properties Mapping
{}
Labels-Properties Mapping
{}

Validate Method

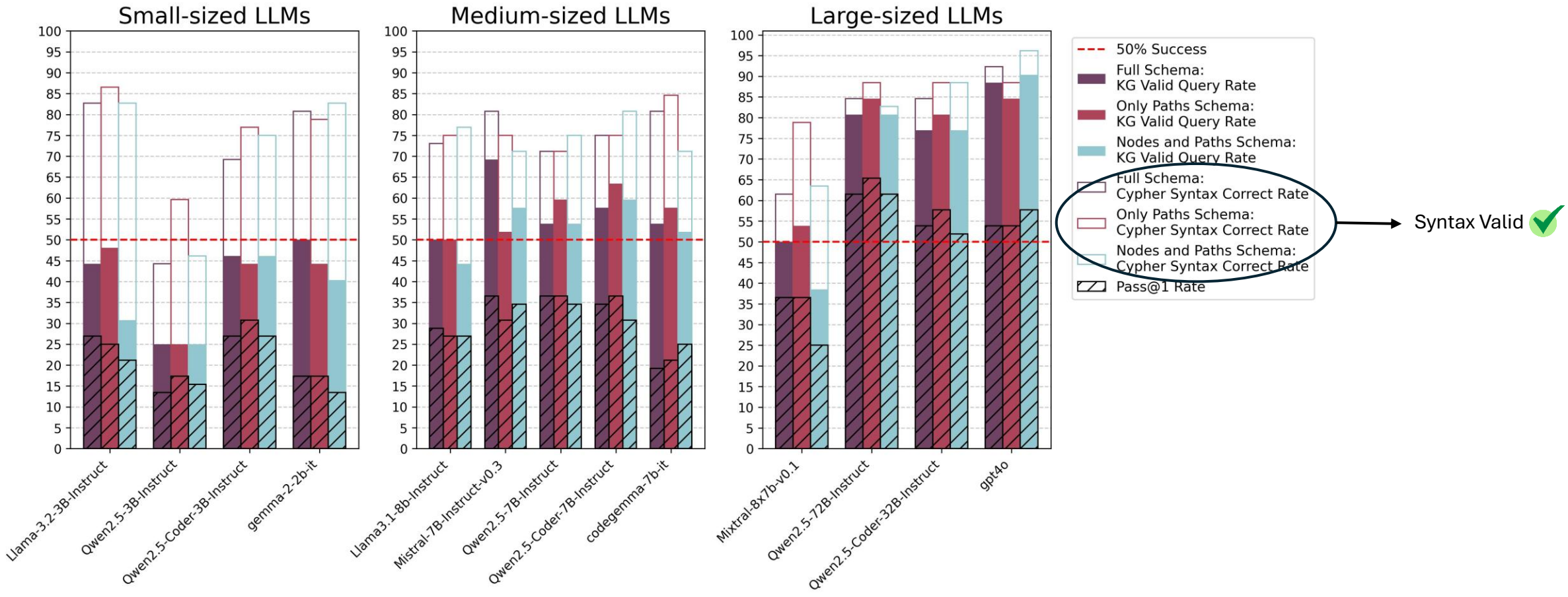
None

Decoding the Mystery: How Can LLMs Turn Text Into Cypher in Complex Knowledge Graphs?



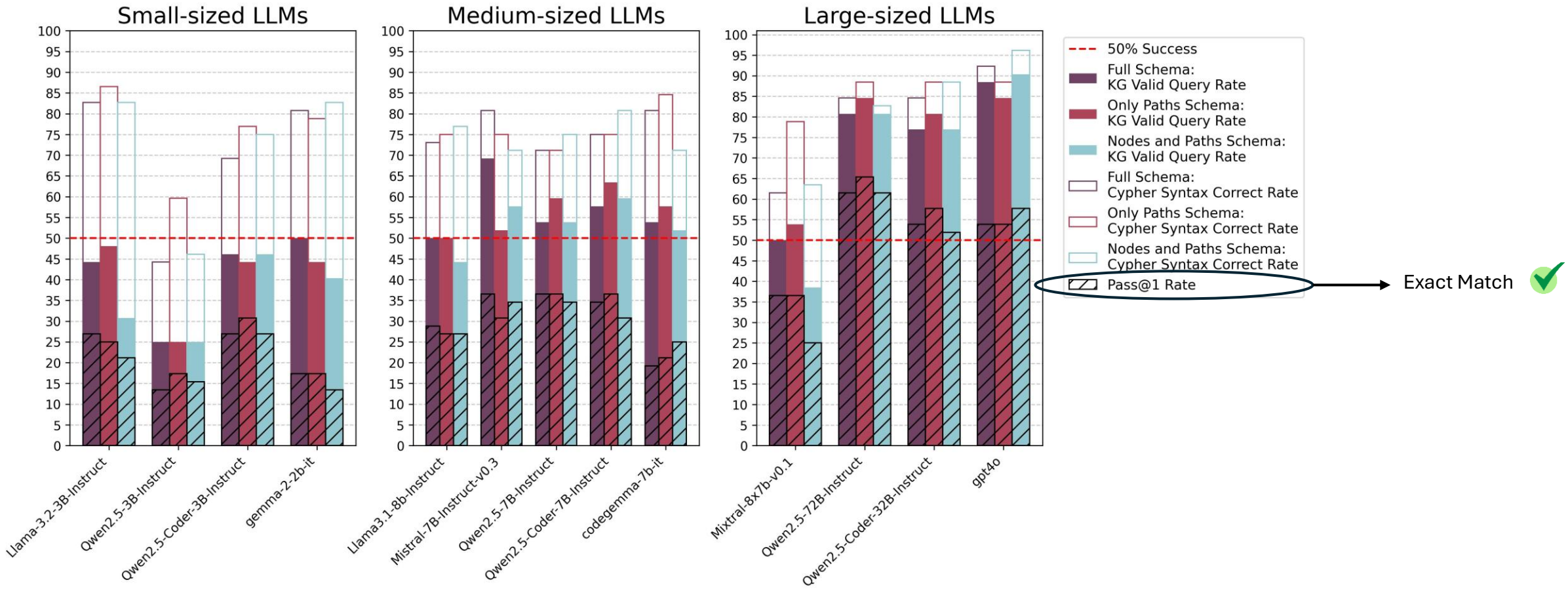
Decoding the Mystery: How Can LLMs Turn Text Into Cypher in Complex Knowledge Graphs? | IEEE Journals & Magazine | IEEE Xplore

Decoding the Mystery: How Can LLMs Turn Text Into Cypher in Complex Knowledge Graphs?



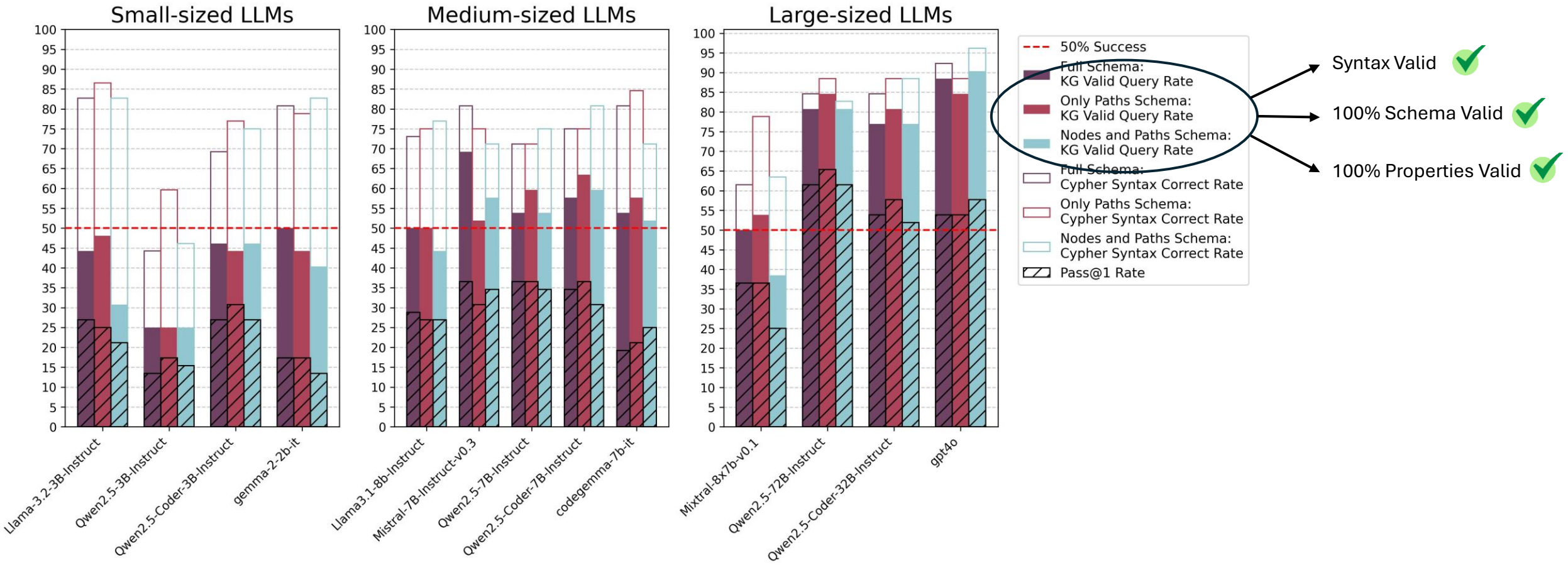
The KG Valid Query Rate, Cypher Syntax Correct Rate and Pass@1 Rate across the LLMs grouped by size.

Decoding the Mystery: How Can LLMs Turn Text Into Cypher in Complex Knowledge Graphs?



The KG Valid Query Rate, Cypher Syntax Correct Rate and Pass@1 Rate across the LLMs grouped by size.

Decoding the Mystery: How Can LLMs Turn Text Into Cypher in Complex Knowledge Graphs?

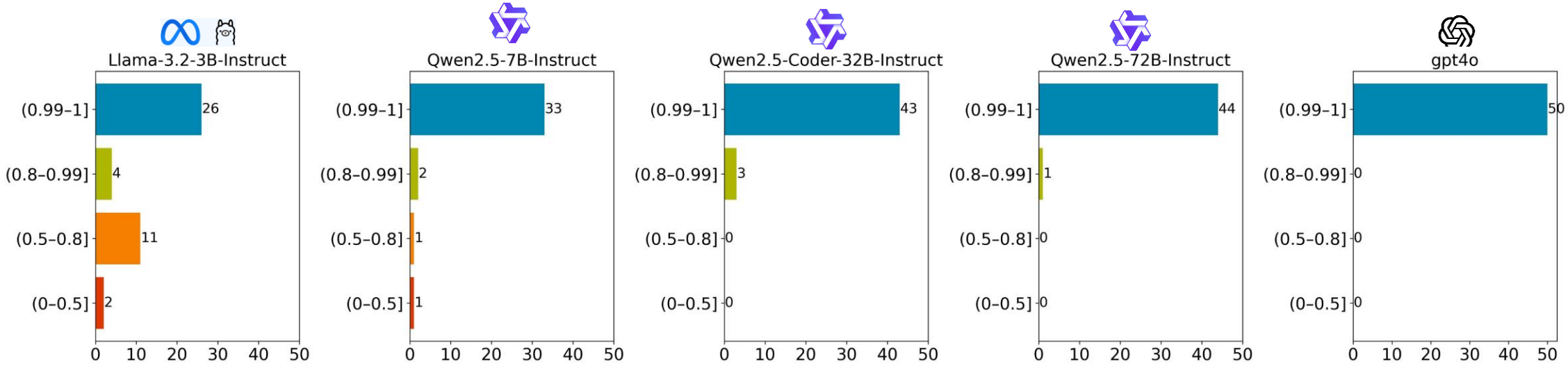


The KG Valid Query Rate, Cypher Syntax Correct Rate and Pass@1 Rate across the LLMs grouped by size.

Where do LLMs fail in Cypher?

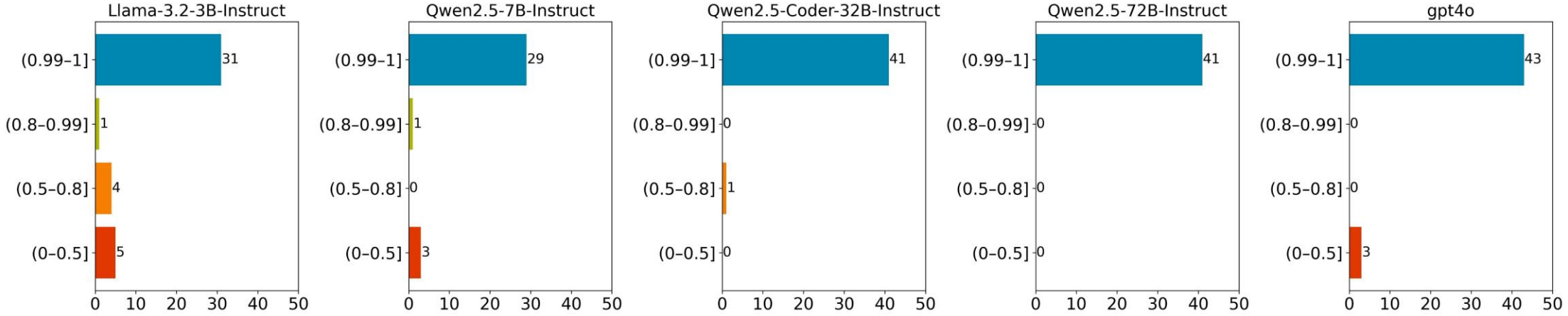
Model size +

Schema Validator Values



The distribution of syntactically correct generated queries across different Schema Validator value ranges in LLMs.

Properties Validator Values



The distribution of syntactically correct generated queries across different Properties Validator value ranges in LLMs.

Where do LLMs fail in Cypher?

Schema
Validator
Values

Hallucinating Labels?

Wrong Direction Path?

Undefined Paths?



Properties
Validator
Values

Hallucinating Properties?

Wrong Label-Properties pairs?

Introducing CyVer v2

Naïve RAG on graph databases

single shot query

Agentic AI systems

feedback on generated queries

self-correction loops

retries to refine



Judge LLM

From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge

vs



Trustworthy?
Hallucinations



Deterministic
Executable-aware
Error metadata

What's new in CyVer v2?

- **Explainability** on Syntax, Schema and Properties validation score with **error metadata**
- **Suggestions** on how to fix the errors

- **Leveraging neo4j Notifications, Warnings, and Error codes**
- **Introducing new Errors and Warnings**

CyVer Error Categories

Syntax validation

1. Execution Failed
2. Syntax Error
3. Parameter not Provided
4. Unsatisfiable Relationship Type Expression
5. Arithmetic Error (explicit division by zero)
6. Variables with conflicting Labels

CyVer Error Categories

Schema validation

1. Unknown Label
2. Unknown Relationship Type
3. Unknown Path
4. Wrong Direction Path
5. Variable Length Path

Properties validation

1. Unknown Property
2. Invalid Property Access by Label



Let’s try 

Thank you!
Questions?

